

INTRODUCCIÓN

- Contexto general del problema (sector, tendencia, necesidad)
- Importancia del problema (impacto real, interés académico/industrial)
- Motivación personal y/o académica
- Problema concreto que se quiere resolver
- Breve descripción de la solución propuesta
- Alcance del proyecto (qué incluye y qué no)
- Estructura del documento (incluyendo anexos)

OBJETIVOS (Sacar del anteproyecto)

Objetivo general

- Qué se pretende conseguir con el TFG (resultado final)

Objetivos específicos

- Lista de objetivos desglosados
- Orientados a:
 - Funcionalidad (qué debe hacer el sistema)
 - Calidad (rendimiento, seguridad, usabilidad...)
 - Técnicos (arquitectura, tecnologías)
 - De investigación (si aplica)

ESTADO DEL ARTE

- Definición de conceptos clave
- Revisión de soluciones existentes:
 - Herramientas similares
 - Productos comerciales / open source
- Comparativa entre soluciones:
 - Ventajas / limitaciones
- Tecnologías relevantes:
 - Frameworks, arquitecturas, paradigmas
- Normativas / estándares (si aplica)
- Justificación de la propuesta:
 - Qué aporta frente a lo existente

METODOLOGÍA

Gestión del proyecto (obligatorio)

- Metodología utilizada (Scrum, Kanban, híbrida...)
- Justificación de la elección
- Adaptación de la metodología al TFG:
 - Número de sprints
 - Duración
 - Roles (aunque sea individual)
 - Adaptación de las sesiones (dailys, semanales, retros, revisión con cliente)
- Herramientas de gestión (Jira, Trello...)

Metodología de desarrollo (obligatorio)

- Enfoque de desarrollo:
 - Iterativo / incremental/ prototipado ...
- Gestión de requisitos
- Gestión de versiones (Git, ramas, GitFlow, ...). Puede estar aquí o en resultado

Stack tecnológico (obligatorio)

- Lenguajes, Frameworks, Herramientas, Infraestructura (cloud, local...)

Planificación (puede estar aquí o en resultados)

- Roadmap del proyecto
- Diagrama de Gantt (muy muy recomendable)
- Backlog inicial:
 - Historias de usuario / requisitos

Estimación (puede estar aquí o en resultados)

- Técnicas utilizadas (story points, horas...)
- Estimación inicial vs real

Estimación de costes y gestión de riesgos del proyecto: Puede estar aquí, en resultados o en anexos. Cálculo realista de costes (salarios, licencias de software, servicios, etc.). Listado de riesgos, probabilidad de ocurrencia, acciones preventivas/correctivas, etc.

RESULTADOS (LA CLAVE, ES LO MÁS IMPORTANTE)

Este capítulo describe el proceso de desarrollo real. Se recomienda iniciar con una definición global de requisitos (Historias de Usuario y Casos de Uso) y luego estructurar cada **Iteración/Sprint** de la siguiente forma:

Estructura FIJA por iteración (Sprint)

Cada iteración debe tener SIEMPRE:

Planificación del sprint

- Objetivo del sprint
- Historias de usuario seleccionadas
- Tareas asociadas
- Estimación inicial:
 - Horas / story points
- Dependencias o riesgos detectados

Trabajo realizado (QUÉ se ha hecho)

- Funcionalidades desarrolladas
- Cambios respecto a lo planificado
- Elementos implementados:
 - Backend
 - Frontend
 - Integraciones
- Decisiones tomadas

Cómo se ha hecho (CÓMO): Aquí es donde se demuestra el nivel técnico. Incluir evidencias como:

- Diseño:
 - Diagramas UML (clases, secuencia, casos de uso...)
 - Arquitectura
- Base de datos:
 - Modelo relacional / ER
- UI/UX:
 - Mockups
 - Prototipos
 - Pantallas

- Código:
 - Fragmentos relevantes explicados
- Integraciones:
 - APIs
 - Servicios externos
- Investigación de tecnologías:
 - Pruebas de tecnologías
 - Comparativas técnicas
- Pruebas:
 - Unitarias, de integración, UAT, etc.

Retrospectiva (MUY IMPORTANTE)

Tiene que ser REALISTA (esto marca la diferencia porque en los proyectos reales las cosas no van siempre bien)

- Qué se ha conseguido realmente
- Qué queda pendiente
- Estado del producto tras el sprint
- Qué ha ido bien
- Qué ha ido mal
- Problemas encontrados:
 - Técnicos
 - De planificación
- Comparación estimación vs realidad
- Gráficas (muy recomendable):
 - Burndown / Burnup
- Desviaciones:
 - Por qué han ocurrido
- Lecciones aprendidas
- Acciones de mejora para siguientes sprints

CONCLUSIONES Y TRABAJOS FUTUROS

Revisión de objetivos: Revisión punto por punto de los objetivos iniciales para confirmar su cumplimiento.

- Revisión de los objetivos:
 - Qué se ha cumplido y qué no y como
- Valoración global del proyecto
- Limitaciones encontradas
- Aportación del proyecto:
 - Técnica
 - Académica
 - Profesional
- Aprendizaje adquirido

Trabajos futuros Listado de mejoras o funcionalidades que quedaron fuera del alcance pero que podrían implementarse.

- Mejoras posibles
- Nuevas funcionalidades
- Escalabilidad del sistema
- Líneas de investigación abiertas

Competencias adquiridas (en función de la intensificación): Justificación de las capacidades de ingeniería desarrolladas durante el proyecto. Tienen que ser conforme al plan de estudios

- Técnicas:
 - Desarrollo, arquitectura, datos...
- Metodológicas:
 - Scrum, planificación...
- Transversales:
 - Resolución de problemas
 - Trabajo autónomo

Valoración Personal: Reflexión subjetiva sobre el aprendizaje y la experiencia.

BIBLIOGRAFÍA Y ANEXOS

Bibliografía: Citación correcta de todas las fuentes bajo normativa (APA o similar).

Anexos (EJEMPLOS)

- Diagramas completos
- Listados de código fuente relevante
- Estudio de viabilidad
- Manual de usuario
- Casos de prueba (más en detalle)
- Backlog completo (si no cabe en la memoria)
- Plan de riesgos identificados
- Costes / viabilidad (si aplica)